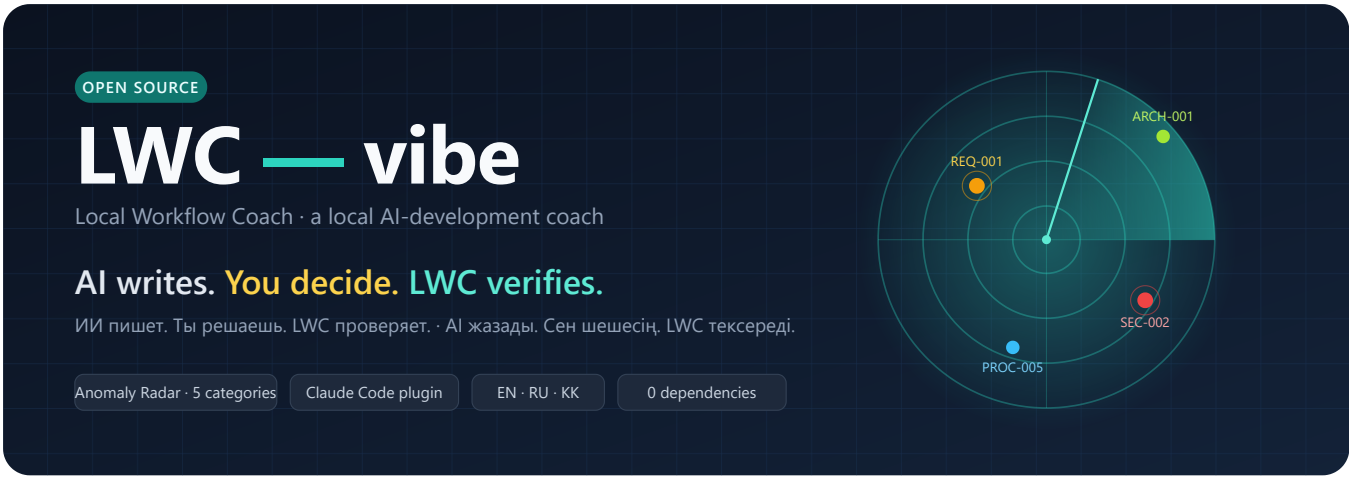


LWC — vibe

AI writes. You decide. LWC verifies.

Local Workflow Coach — a local AI-development coach with Anomaly Radar: an open-source tool for developers and teams who use AI to write software and want to keep their understanding of the system and control over the result.

Technical documentation · version 0.2.0 · 20 October 2026
Repository: github.com/RuslanKazyradzi13/lwc-vibe · Licence: MIT · Author: Ruslan Kazyradzi

Contents

1. Purpose
2. Differences from the reference (VibeWise)
3. Architecture
4. Workflow
5. Anomaly Radar
6. Anomaly examples
7. Severity scale
8. Report example with explanations
9. Evidence Gate and the optional Nace advisory
10. Operational report check (Kazakhstan)
11. Installation
12. Run example
13. Security and limitations
14. Licensing
15. Roadmap

Reports are available in English, Russian and Kazakh. Қазақ тілі толық қолдау табады (ә, ғ, қ, ң, ө, ұ, ү, h, і). Русский и казахский — полные каталоги сообщений.

1. Purpose

An AI agent can write an application in an evening that its author cannot explain, let alone verify. LWC — vibe helps you use AI to write software without losing understanding of the system and control over the result. The agent clarifies requirements first, shows a plan, discusses risks, gets explicit confirmation for significant decisions, implements small changes and verifies the result. Anomaly Radar checks the agent's words against facts: the import graph, the Git state, records of real command runs.

The product has two parts: a **dependency-free CLI** (Node.js ≥ 20, works in a terminal and in CI) and a **Claude Code plugin** with seven modes (Learn, Design, Build, Review, Explain, Anomaly Radar, Resume) and two hooks — context restoration at session start and a guard against writing code before the plan is confirmed.

Three explanation levels — beginner, intermediate, advanced — change the depth of explanations and the step size, but not the degree of control: important architectural decisions and potentially dangerous changes are always confirmed by the user.

2. Differences from the reference (VibeWise)

The project is inspired by the VibeWise approach (Noah Kim, MIT, 2026) — "learn while AI writes the code" — but it is an independent implementation: the reference's code, texts, illustrations and structure were not used.

	VibeWise	**LWC — vibe**
Control over code before agreement	an instruction in a Markdown skill	a task stage in .lwc/session.json, the CLI gate `lwc task stage build`, a PreToolUse hook (ask/deny), rule PROC-001
"Tests pass"	the agent's words	`lwc run` records the real run; claims are checked against evidence (PROC-002, PROC-005)
Checking requirements, architecture, code, security	none	Anomaly Radar: 5 categories, 29 rules, a status and a source on every finding
Operational data	none	`lwc ops-check` with configurable rules, an example in KZT
Languages	English	English, Russian, Kazakh
Works without an agent	no	yes: a CLI for the terminal and CI

3. Architecture

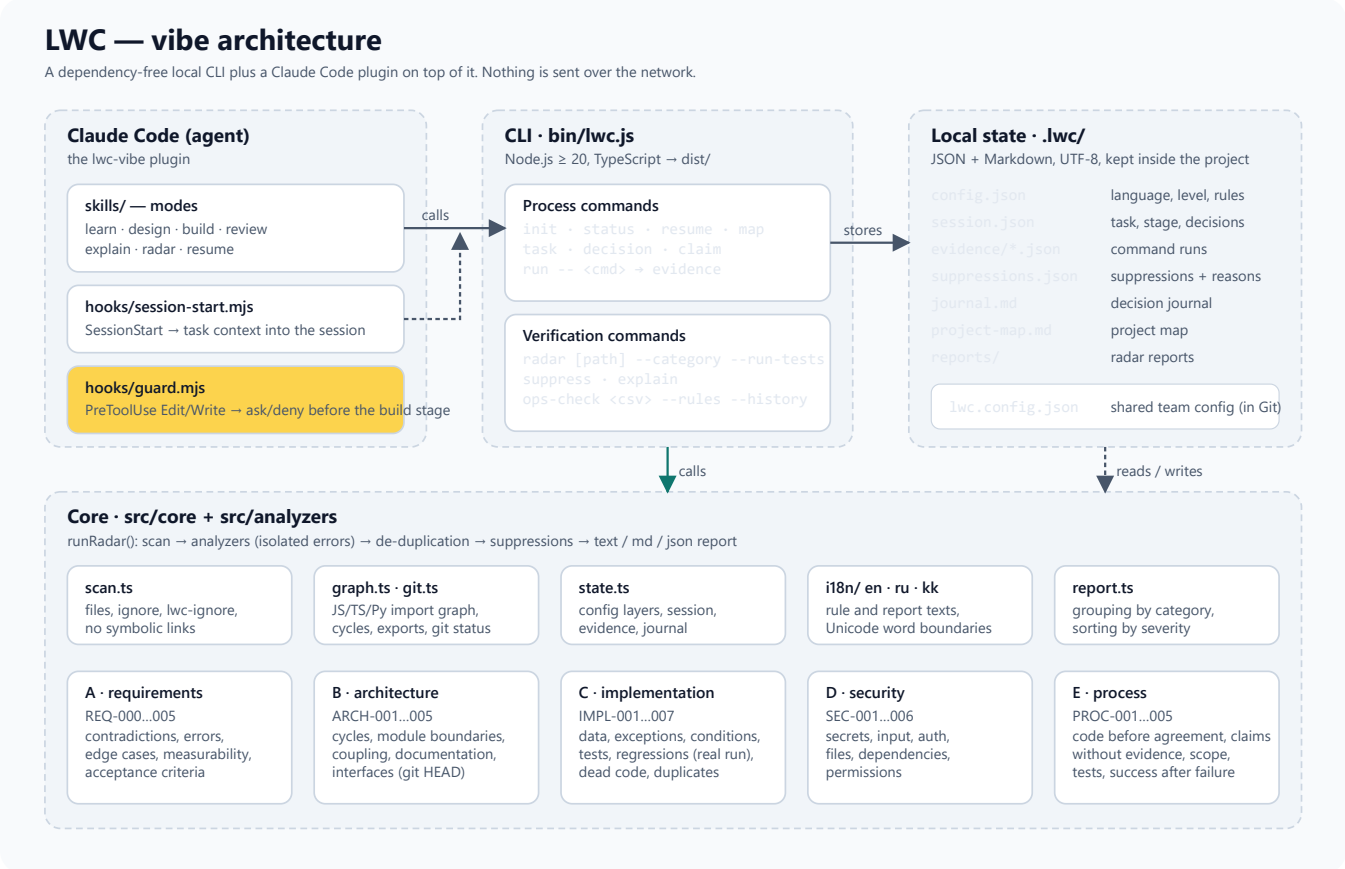


Figure 1. A local CLI (bin/lwc.js → dist/) and a Claude Code plugin on top of it; state is JSON and Markdown in the .lwc/ directory.

The core (`src/core`) handles file scanning, the JS/TS/Python import graph, Git access, configuration layers (`DEFAULT ← lwc.config.json ← .lwc/config.json`) and report rendering. Analyzers (`src/analyzers`) are pure functions from a context to a list of findings; one analyzer's error becomes a `RADAR-ERR` finding and the others keep running. Project commands are executed only on explicit request (`lwc run` , `--run-tests`).

4. Workflow

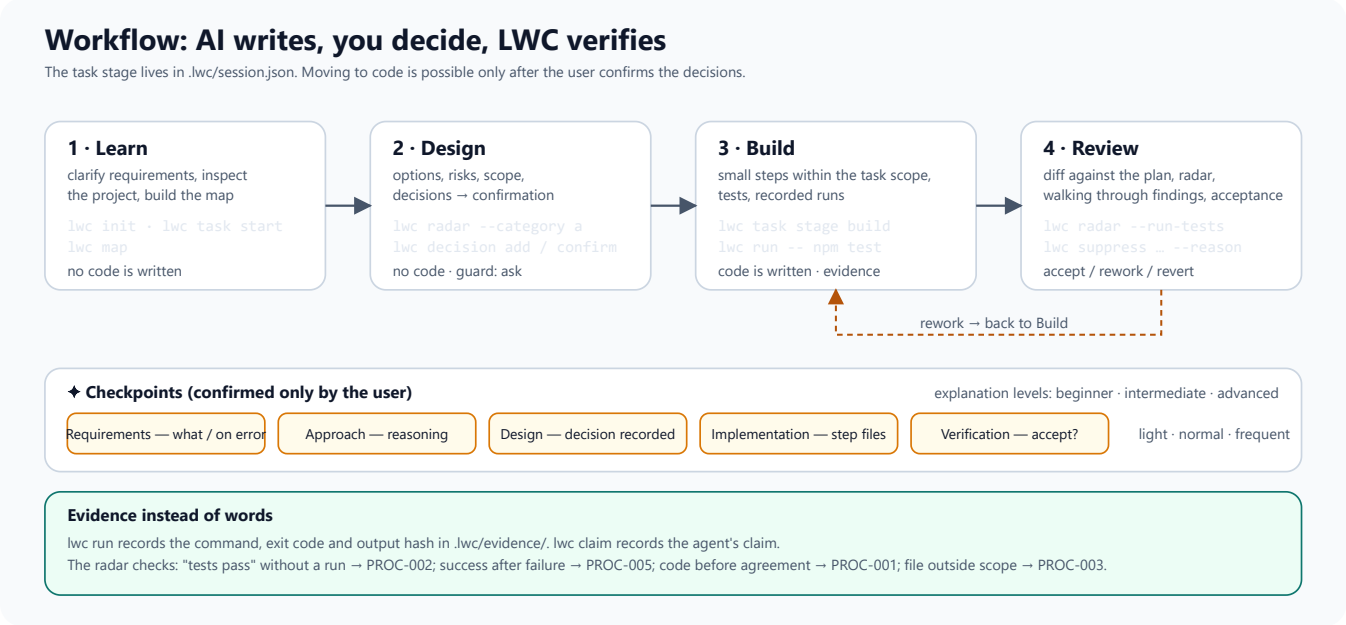


Figure 2. The stages Learn → Design → Build → Review, checkpoints and evidence.

Mode	Code written?	What happens
Learn	no	clarifying requirements, inspecting the project, the map (`lwc map`)
Design	no	options, risks, scope, decisions → user confirmation (`lwc decision confirm`)
Build	yes, within the task scope	small steps, tests, runs through `lwc run`, claims through `lwc claim`
Review	no	diff against the plan, `lwc radar --run-tests`, walking through findings, acceptance
Explain / Radar / Resume	no	explanations based on real module relations; anomaly search; context restoration

5. Anomaly Radar

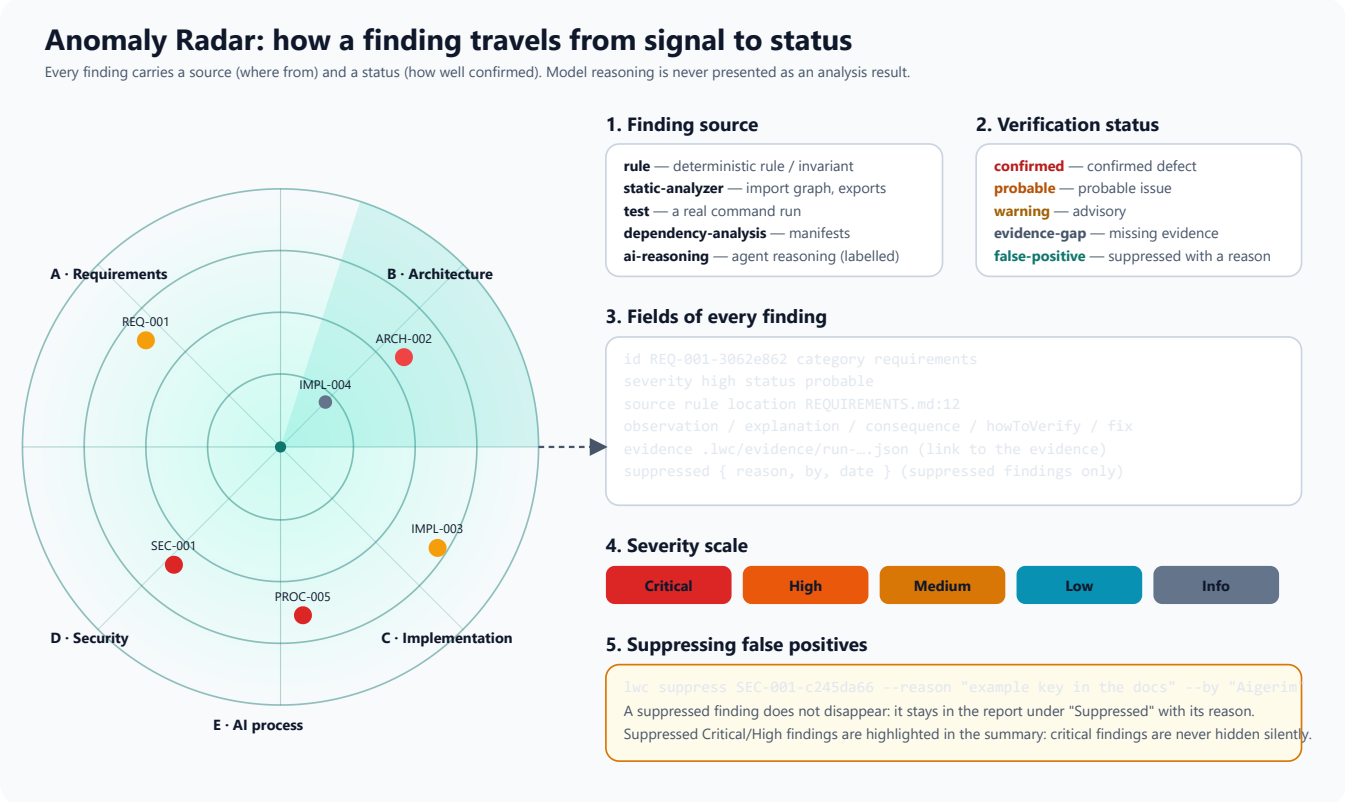


Figure 3. The path of a finding: source → status → fields → severity → suppression with a reason.

Every finding carries a **source** (rule, static analysis, test run, dependency analysis, AI agent reasoning) and a **status** (confirmed defect, probable issue, warning, evidence gap, false positive). The radar never presents model reasoning as a static-analysis or test result. False positives are suppressed only with a reason and stay in the report; suppressed Critical/High findings are highlighted in the summary.

6. Anomaly examples

Category	Rules	Examples
A. Requirements	REQ-000…005	two requirements contradict each other; error behaviour is undefined; edge cases are missing; a requirement cannot be verified ("fast and intuitive"); a business rule is missing from the acceptance criteria
B. Architecture	ARCH-001…005	cyclic dependencies; a declared module boundary is violated; excessive coupling; documentation diverges from the structure; an interface changed without updating dependents
C. Implementation	IMPL-001…007	JSON.parse without try; an empty catch; `i <= arr.length`; assignment in if; missing tests; a regression in a real test run; an unused export; duplication
D. Security	SEC-001…006	secrets in code; SQL/shell injection, eval, XSS; a route without authorisation, disabled TLS; a path from user input; suspicious dependencies; excessive permissions
E. AI-development process	PROC-001…005	code before agreement; "verified" without a run; a file outside the scope; changes without tests; success claimed after a failed check

7. Severity scale

Level	Meaning	Example
Critical	exploitable vulnerability, data loss, a false "success" after a failure	an AWS key in code, SQL injection, PROC-005
High	the bug will almost certainly show, or an agreement is broken	a three-module cycle, `i <= length`, code before agreement
Medium	a probable bug or a gap that needs a decision	JSON.parse without try, total ≠ sum of parts
Low	quality and maintenance, weak heuristics	loose ==, an unused export
Info	information needing no action	requirements file not found, analyzer error

8. Report example with explanations

Below is a fragment of the **real output** of `lwc radar examples/demo-buggy-app` on the demo project "coffee-shop checkout" with deliberately planted defects (synthetic data, fictional keys; the requirements file is in Russian to demonstrate Cyrillic analysis). The full list of expected findings is recorded in `examples/demo-buggy-app/EXPECTED_FINDINGS.md` and verified by a test.

```
Anomaly Radar report · lwc-vibe 0.2.0
Project: <repo>\examples\demo-buggy-app
Generated: 2026-10-20T15:50:30.124Z · Files scanned: 20 · Analyzers: requirements, architecture,
implementation, security, process, tests-runner

Summary: Total findings 44
  ■ Critical: 6   ▲ High: 11   ● Medium: 11   ○ Low: 16   · Info: 0


▲ [High] REQ-001-3062e862  Contradiction between requirements
    Where: REQUIREMENTS.md:12
    Observation: Requirement "Скидка по карте лояльности должна применяться ко всем позициям заказа."
contradicts requirement "Скидка по карте лояльности не должна применяться к позициям со специальной
ценой. (line 13)"
    Status: probable issue · Source: rule
    Recommended fix: Keep one requirement or specify the conditions under which each applies.


▲ [High] ARCH-002-ed516db9  Declared module boundary violated
    Where: src/ui/receipt.js:1
    Observation: src/ui/receipt.js imports src/db/storage.js, which boundary rule "the UI never talks to
storage directly — only through services/" forbids
    Status: confirmed defect · Source: static analysis
    Recommended fix: Route the dependency through an interface/abstraction or revise the rule.


■ [Critical] PROC-005-5114eb44  Success claimed although the check failed
    Observation: The last run of `node --test tests/` exited with code 1, yet claim "Implemented, tests
pass" was made afterwards
    Status: confirmed defect · Source: rule · Evidence: .lwc/evidence/run-20261019123000-a1b2c3.json
    Recommended fix: Fix the problem and rerun the check before claiming success.

Suppressed: 2
```

How to read it: the first line is the severity, identifier and title; *Where* — file and line; *Observation* — the fact (a quote); *Status* and *Source* — how well the finding is confirmed and where it came from; *Evidence* — a link to the run record; *Recommended fix* — what to do. The Markdown and JSON formats additionally include the plain-language explanation, the possible consequence and how to verify.

Report example: lwc radar examples/demo-buggy-app

A fragment of the real output (text format, English). Full format — docs/REPORT_FORMAT.md; md/json examples — examples/.

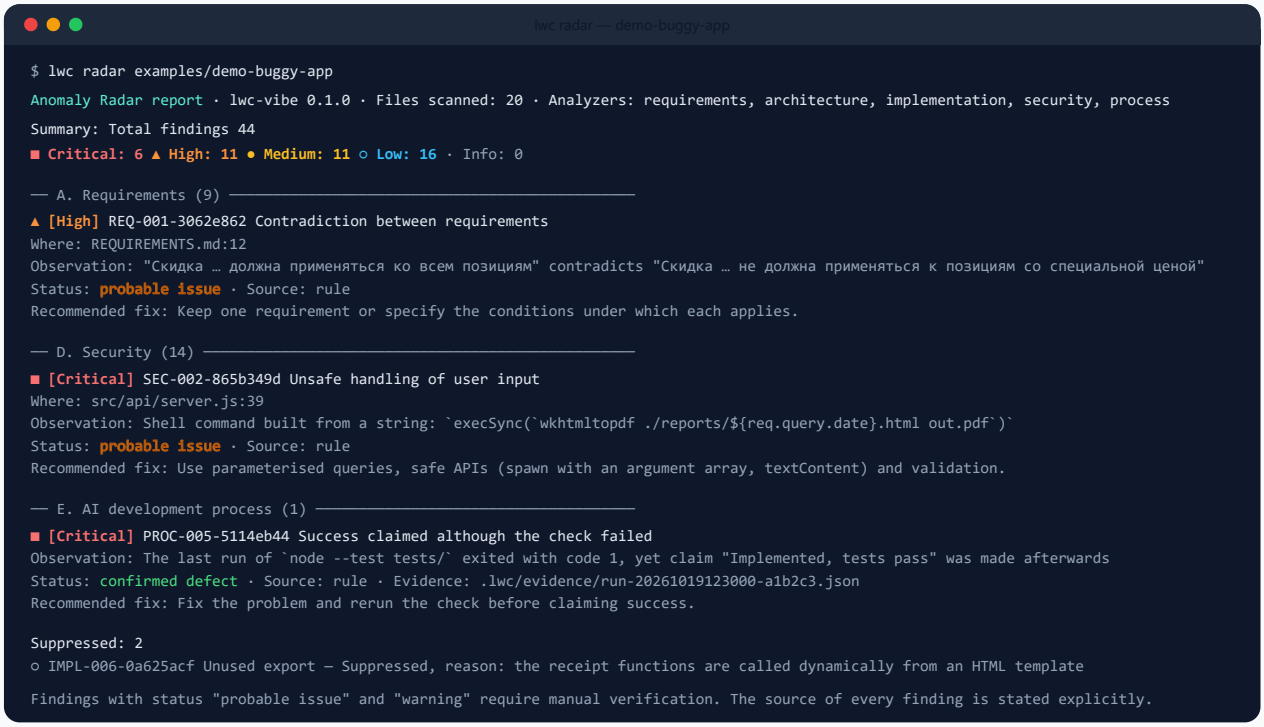


Figure 4. A styled fragment of the same report (in the text above the repository path is replaced with <repo>).

9. Evidence Gate and the optional Nace advisory

An agent's "done, tests pass" is a claim, not a fact. `lwc gate` checks whether the recorded facts support it **for the code as it is now**. Every `lwc run` stores a fingerprint of the working tree, so a test run made before the latest edit is *stale* and proves nothing. Claims are matched to the evidence they need. Security and requirement claims are marked as not verifiable automatically. Five checks give the verdict **PASS**, **INSUFFICIENT EVIDENCE** or **FAIL**, with exit codes for CI.

Check	fail	gap
GATE-TESTS	the latest test run failed	no run or a stale run while tests are required
GATE-CLAIMS	a claim is contradicted	a claim is unsupported, stale or unverifiable
GATE-FINDINGS	a confirmed Critical/High finding	a Critical/High finding that needs manual verification
GATE-DECISIONS	—	unconfirmed decisions at the build/review stage
GATE-ANALYZERS	—	an analyzer crashed

Real output on the demo project (the claim "Implemented, tests pass" was recorded after a failing run):

```
Evidence Gate · lwc-vibe 0.2.0
Project: <repo>\examples\demo-buggy-app
Task: "Staff discount 20 %" (review)

Verdict: FAIL — the recorded checks show problems that block the task
  The verdict rests on recorded runs and enabled rules only. A pass is not proof of correctness or
security.

── Blocking problems (8) ──────────────────────────────────────
  ✕ GATE-TESTS · Test run on the current code — The last test run `node --test tests/` exited with code
1 [run-20261019123000-a1b2c3]
  ✕ GATE-CLAIMS · Agent claims backed by evidence — 1 of 1 claim(s) are contradicted by the evidence
[C1]
  ✕ GATE-FINDINGS · No confirmed Critical/High findings — 4 confirmed Critical/High finding(s); 13 more
need manual verification [PROC-005-5114eb44, SEC-006-d133194d, SEC-003-cf53ecc4, ARCH-002-ed516db9, SEC-
002-a9109d91, …]
  C1 "Implemented, tests pass" [tests, completion] → contradicted: the run before the claim, `node --
test tests/`, exited with code 1 · Evidence: run-20261019123000-a1b2c3
  [Critical · confirmed defect] PROC-005-5114eb44 Success claimed although the check failed
  [High · confirmed defect] SEC-006-d133194d Excessive permissions (.github/workflows/deploy.yml:3)
  [High · confirmed defect] SEC-003-cf53ecc4 Potential authentication/authorisation issue
(src/api/server.js:45)
  [High · confirmed defect] ARCH-002-ed516db9 Declared module boundary violated (src/ui/receipt.js:1)

── Not confirmed yet (0) ──────────────────────────────────────
  nothing

── Confirmed by deterministic checks (2) ──────────────────────
```

Nace Drex (optional). With `--ai nace` and the `NACE_API_KEY` environment variable, LWC asks Nace's decision model to triage the findings and rate overall readiness. It uses one documented endpoint, `POST /v1/systemone`, over plain `fetch`. The answer is shown in a separate "AI advisory (not evidence)" section. It is computed after the verdict and cannot change it. Only redacted metadata is sent, never source code or command output. `--ai-preview` prints the payload without sending it:

```
{
  "schema": "lwc-gate-advisory/1",
  "task": {
    "title": "Staff discount 20 %",
    "stage": "review"
  },
  "verdict": "fail",
  "claims": [
    {
      "id": "C1",
      "text": "Implemented, tests pass",
      "kinds": [
        "tests",
        "completion"
      ],
      "status": "contradicted"
    }
  ],
  "findings": [
    {
      "ref": "PROC-005-5114eb44",
      "rule": "PROC-005",
      "category": "process",
      "severity": "critical",
      "status": "confirmed",
      "title": "Success claimed although the check failed"
    },
    {
      "ref": "SEC-002-a9109d91",
      "rule": "SEC-002",
      "category": "security",
      "severity": "critical",
      "status": "probable",
      "title": "Unsafe handling of user input",
```


The Nace integration follows the official API reference and is covered by tests against mocked responses. As of 20 October 2026 no call to the real Nace API has been made, because no key was available. docs/NACE.md lists what still has to be checked with a real account.

10. Operational report check (Kazakhstan)

Scenario "anomalies in a daily operational report": a synthetic report of a pickup-point network for one day, amounts in tenge. The rules are JSON and can be adapted per organisation; the historical spike is computed from a history file and a configurable threshold and has the status "probable issue", not an error. Real output:

```
Daily operational report: pickup-point network (synthetic data, KZT) · examples/ops-
report/daily_report.csv · 8 rows · Rules: OPS-008, OPS-001, OPS-002, OPS-003, OPS-004, OPS-005, OPS-007,
OPS-006
Total findings: 7

[High] OPS-004-5d7c7d47 Duplicate record
    Observation: Rows 3, 8 share the same key (point_id+date = ALA-02|2026-10-18)
    Explanation: Two records have the same uniqueness key defined in the rules.
    Status: confirmed defect · Source: rule
    Recommended fix: Remove the duplicate or refine the uniqueness key if these are different
operations.

[High] OPS-001-9001fb9c Required value missing
    Observation: Row 4: field "orders_count" is empty
    Explanation: The field is declared required in the report rules, but the value is missing.
    Status: confirmed defect · Source: rule
    Recommended fix: Fill in the value at the source or exclude the row with a note.

[High] OPS-002-b8bec827 Negative value where forbidden
    Observation: Row 6: "refunds_kzt" = -4 000 ₸
    Explanation: A quantity or amount of this type cannot be negative under the report rules.
    Status: confirmed defect · Source: rule
    Recommended fix: Fix the sign or record the operation with the correct type (refund/adjustment).

[High] OPS-003-9ca7592b Total does not match the sum of parts
    Observation: Row 6: "revenue_kzt" = 1 210 000 ₸, sum of cash_kzt + card_kzt = 1 160 000 ₸
(difference 50 000 ₸)
    Explanation: The total field must equal the sum of its parts within the configured tolerance.
    Status: confirmed defect · Source: rule
    Recommended fix: Recalculate the total or fix the parts at the source.

[High] OPS-005-8243a3b1 Impossible time sequence
    Observation: Row 7: "closed_at" (2026-10-18 09:00) is earlier than "opened_at" (2026-10-18 21:15)
    Explanation: By the rules, event "closed_at" cannot happen before event "opened_at".
    Status: confirmed defect · Source: rule
    Recommended fix: Fix the timestamps; align all sources to one timezone.

[Medium] OPS-006-388a48a3 Sharp spike relative to history
    Observation: "revenue_kzt (sum)" = 28 402 000 ₸, median of 7 previous periods = 8 360 000 ₸ (ratio
3.40, threshold ×2.5)
    Explanation: The value differs strongly from history. Not necessarily an error: the rule only flags
the value for manual review using a configurable threshold.
    Status: probable issue · Source: rule
    Recommended fix: Confirm or correct the value; adjust the threshold in the rules if needed.

[Medium] OPS-007-4bc1f67b Value outside the allowed range
    Observation: Row 9: "avg_ticket_kzt" = 300000, allowed [500; 50000]
    Explanation: The rule defines an allowed range for the field; the value is outside it.
    Status: confirmed defect · Source: rule
    Recommended fix: Fix the value or revise the range in the rules.
```

The same report in Russian (`--lang ru`) and Kazakh (`--lang kk`):

```
Daily operational report: pickup-point network (synthetic data, KZT) · examples/ops-report/daily_report.csv · 8 строк · Правила: OPS-008, OPS-001, OPS-002, OPS-003, OPS-004, OPS-005, OPS-007, OPS-006
Всего находок: 7

[Высокая] OPS-004-5d7c7d47 Дублирующаяся запись
    Наблюдение: Строки 3, 8 совпадают по ключу (point_id+date = ALA-02|2026-10-18)
    Объяснение: Две записи имеют одинаковый ключ уникальности, заданный в правилах.
    Статус: подтверждённый дефект · Источник: правило
    Рекомендуемое исправление: Удалите дубликат или уточните ключ уникальности в правилах, если это разные операции.

[Высокая] OPS-001-9001fb9c Отсутствует обязательное значение
    Наблюдение: Строка 4: поле «orders_count» пустое
    Объяснение: Поле объявлено обязательным в правилах отчёта, но значение отсутствует.
    Статус: подтверждённый дефект · Источник: правило
    Рекомендуемое исправление: Заполните значение в источнике данных или исключите строку с пометкой.
```

```
Daily operational report: pickup-point network (synthetic data, KZT) · examples/ops-report/daily_report.csv · 8 жол · Ережелер: OPS-008, OPS-001, OPS-002, OPS-003, OPS-004, OPS-005, OPS-007, OPS-006
Барлық табылғандар: 7

[Жоғары] OPS-004-5d7c7d47 Қайталанатын жазба
    Байқау: 3, 8 жолдары кілт бойынша сәйкес келеді (point_id+date = ALA-02|2026-10-18)
    Түсіндірме: Екі жазбаның ережелерде берілген бірегейлік кілті бірдей.
    Күйі: расталған ақау · Дереккөз: ереже
    Ұсынылатын түзету: Қайталануды жойыңыз немесе бұл әртүрлі операциялар болса, ережелердегі бірегейлік кілтін нақтылаңыз.

[Жоғары] OPS-001-9001fb9c Міндетті мән жоқ
    Байқау: 4-жол: «orders_count» өрісі бос
    Түсіндірме: Өріс есеп ережелерінде міндетті деп жарияланған, бірақ мән жоқ.
    Күйі: расталған ақау · Дереккөз: ереже
    Ұсынылатын түзету: Дерек көзінде мәнді толтырыңыз немесе жолды белгімен алып тастаңыз.
```

11. Installation

Requirements: Node.js 20 or newer; Git — for category E rules and ARCH-005.

Claude Code plugin (inside a Claude Code session, one command at a time):

```
/plugin marketplace add RuslanKazyradzi13/lwc-vibe
/plugin install lwc-vibe@lwc-vibe
```

CLI:

```
npm install -g github:RuslanKazyradzi13/lwc-vibe
lwc --version      # 0.2.0
```

Or from a clone: `git clone … && cd lwc-vibe && npm install && npm link`. Updating: `git pull` in the clone, or `claude plugin marketplace update lwc-vibe` and `claude plugin update lwc-vibe@lwc-vibe`.

12. Run example

```
cd my-project
lwc init --lang en --level beginner
lwc task start "Staff discount 20 %" --scope "src/services/**,tests/**"
lwc decision add "The discount is a separate function in discount.js"
lwc task stage build          # refuses: decision D1 is not confirmed
lwc decision confirm D1       # only a human confirms
lwc task stage build
lwc run -- npm test           # the run is recorded in .lwc/evidence/
lwc claim add "Implemented, tests pass"
lwc radar --run-tests --format md --out .lwc/reports/review.md
lwc gate                      # is claim C1 backed by a run on the current code?
lwc task finish
```

CLI help:

```
LWC — vibe 0.2.0 · Local Workflow Coach
AI writes. You decide. LWC verifies.

Usage: lwc <command> [options]

State and process
  init [--lang ru|kk|en] [--level beginner|intermediate|advanced] [--checkpoints light|normal|frequent]
[--test-command "npm test"]
  status [--json]                    current task, stage, decisions, last run
  resume [--json]                    context for restoring an agent session
  map                                refresh the project map (.lwc/project-map.md)
  task start "title" [--scope "src/**,tests/**"]
  task scope add|set|clear <glob...>
  task stage learn|design|build|review|done [--yes]
  task finish
  decision add "text" | confirm <id> | list
  claim add "text" | list            record an agent claim (checked against evidence)
  run [--kind test|lint|build] -- <command>   run a command and record evidence
```

```
Anomaly Radar
  radar [path...] [--format text|md|json] [--category a,b,c,d,e] [--run-tests] [--out file] [--fail-on
high] [--lang ru|kk|en] [--verbose]
  suppress <id | RULE-ID> [--file path] --reason "…" [--by name] | suppress list
  explain <file>                      module relations: what it imports, who imports it

Evidence Gate
  gate [path] [--format text|md|json] [--out file] [--run-tests] [--fail-on fail|insufficient|never]
       [--ai nace|off] [--ai-preview] [--lang ru|kk|en]
                                      is the task really done? checks claims against recorded
                                      evidence; optional Nace Drex advisory (never part of the verdict)

Operational data
  ops-check <data.csv|json> --rules <rules.json> [--history <history>] [--format text|md|json] [--lang
ru|kk|en] [--out file]

General
  --root <dir>   project root (default: nearest directory with .lwc or .git)
  --help, --version
```

13. Security and limitations

- Everything runs locally: no telemetry, accounts or paid backend. The only possible network call is the optional Nace advisory, made only when enabled and with `NACE_API_KEY` set. API keys are never read from config files. Project commands are executed only on explicit request.
- An AI answer never counts as evidence. "Nace found no risk" is never reported as secure.
- Hooks never block the session on error; the guard hook asks by default rather than blocking.
- `.lwc/` files contain fragments of code and command output — they are project working files.
- Category A, C and D rules are regular-expression heuristics, not AST analysis: misses and false positives are possible; the finding status reflects this.
- Code is analysed for JavaScript/TypeScript and Python; other languages — secrets and configuration only. The import graph does not know path aliases or monorepos.
- Only the Claude Code integration is tested. Other agents are not claimed.
- Claim classification is keyword-based (English, Russian, Kazakh). Run freshness needs git; without it, freshness is reported as unknown.

> LWC — vibe does not claim compliance with the legislation of the Republic of Kazakhstan or any other country and does not check it. If a project processes personal or financial data, study the primary sources and record the date of the check. As of 20 October 2026 the authors have not performed a legal review.

14. Licensing

LWC — vibe code is distributed under the MIT licence (© 2026 Ruslan Kazyradzi). There are no runtime dependencies; development dependencies are TypeScript (Apache-2.0), Prettier (MIT) and @types/node (MIT). The VibeWise reference (MIT, Noah Kim) was studied as a product idea; its materials were not copied. The full list is in THIRD_PARTY_NOTICES.md.

15. Roadmap

1. The Evidence Gate as a GitHub Action posting its Markdown report to pull requests; gate history per task and per agent.
2. Nace Perception for extracting acceptance criteria from requirement documents.
3. AST analyzers for JS/TS based on the TypeScript Compiler API — fewer false positives, more rules.
4. A PostToolUse hook that records test runs from Bash into `.lwc/evidence/` automatically.
5. Path aliases and monorepos in the import graph; SARIF export for GitHub Code Scanning.
6. Skills for other agents on top of the same CLI — only after real testing.
7. Editorial review of the Kazakh catalog by native speakers.

This document was built automatically by `docs/build-pdf.mjs` from the repository sources on 20 October 2026. The output fragments are the result of actually running the commands while building the document.